The M.U.G.E.N syntax database

The code editors (definitions, commands, states, storyboards, fonts and screenpacks) learn the M.U.G.E.N language from data files in data/engines/mugen. This page explains those files, so you can fix them or make your own versions. Users can replace any of them with a copy in <userdir>/engines/mugen (see the Engine plugins document).

File	Used for
syntax.xml	Highlighting, checking the code, version checks, looking up parameters and triggers
completer.xml	The completion lists shown after type=, const(and similar text
snippets/*.xml	Code snippets with input fields (Code toolbar)
sctrls/*.sctrl	State controller templates (the list in the States editor), made from syntax.xml
palFX/*.xml	Templates for the PalFX editor
docs/<lang>/...	The reference pages that open with Ctrl+click on a controller or trigger

These are the same formats Fighter Factory Studio 3.7 uses for its engine data, so your own files made for it load as they are. The few things added here are listed under Extensions at the end. Programs that don't know them can ignore them.

syntax.xml

```
<syntax>
  <file type="state">
    <block type="State>ChangeState">
      <param name="value" type="expi" required="true"/>
      <param name="ctrl" type="expb"/>
    </block>
    ...
  </file>
  ...
  <consts name="const"> <const name="data.life"/> ... </consts>
  ...
  <trigger name="Pos" type="keyword" list="X,Y" context="default"/>
</syntax>
```

First come the <file> elements, then the <consts> lists, then the <trigger> entries. Upper and lower case don't matter in the names of blocks, parameters, constants, triggers and keywords.

File types

Each <file type="..."> describes one kind of file. The editor picks one from the project type and the file extension.

type	Files
char	Character .def: [Info], [Files], [Palette Keymap], [Arcade]

type	Files
state	.cns and .st: the constants ([Data], [Size], [Velocity], [Movement], [Quotes]), [Statedef] and every state controller
command	.cmd: [Command], [Remap], [Defaults] and the state blocks (for [Statedef -1])
stage	Stage .def: [Info], [Camera], [PlayerInfo], [Bound], [StageInfo], [Shadow], [Reflection], [Scaling], [Music] and the background blocks
fight	fight.def: lifebars, power bars, faces, names, time, combo, round and win icons
select	select.def
motif	system.def: every screen with its backgrounds (TitleBG..., SelectBG... and so on)
font	FNT v2 font .def
storyboard	Storyboard .def: [Info], [SceneDef], [Scene ...] and backgrounds

Blocks

<block type="NAME[:BASE]" [minver="..."] [deprecated="..."] [raw="true"]> describes one section.

- **Plain blocks** (Camera, Select Info, StateDef, Scene) match a section whose header starts with the same words. StateDef matches [Statedef 200] and Scene matches [Scene 3 intro].
- **Typed blocks** like State>HitDef, BG>Parallax or Def>TrueType match a section whose first word is the part before > and whose type parameter is the part after it. [State 210, Kick] with type = HitDef uses State>HitDef, and [BG sky] with type = parallax uses BG>Parallax. The engine only reads the first word of the value (type = truetype font), and outside state controllers it also accepts short forms (type = norm), so a unique start of the word is fine there too.
- **Wildcards**: a * in a block name matches any text, so BG*Def matches [BGDef] and [BG0Def]. If more than one block matches, the one with more fixed letters wins (BG*CtrlDef beats BG*Def for [BG0CtrlDef walk]).
- **Translated sections**: a two-letter language prefix like [ja.Quotes] is ignored.
- **Storyboards**: a scene names its background with bg.name, and the sections <name>Def, <name> ..., <name>CtrlDef and <name>Ctrl ... are checked like the BG* blocks.
- [Begin Action n] sections hold animation data (the AIR format). The animation code reads them, not the database.

Inheriting. NAME:BASE copies every parameter of BASE. The block can add more parameters, and can change an inherited one by defining one with the same name (State>SuperPause:Pause makes the time it inherits optional). BASE is looked up in this order:

1. a block named BASE in the same file type;
2. Section>BASE, with the section of the block that inherits (State>AllPalFX:PalFX);
3. any typed block in the same file type whose type is BASE.

file.BASE looks in another file type in the same way (State>HitDef:state.HitDef, TitleBG>Normal:stage.BG>Normal). Older syntax files name the stage background blocks with dots (stage.BG.Normal, stage.BGCtrlDef.Visible, stage.Dummy). These are read as the matching BG>... and BGCtrl>... blocks of the stage file type. A double colon (::) counts as a single one.

Parameters every block has. Every [State ...] block also accepts type, triggerall, trigger1 to triggerN, persistent and ignorehitpause, and every typed section accepts type. They aren't listed in the blocks.

Parameters

<param name="..." type="..." .../>

Attribute	What it means
name	The parameter's name. A * matches any text (layer*.anim, victory*). A parameter named * accepts anything (State>Null uses it).
type	The kind of value, see below.
required="true"	The section must have this parameter.
optional="req,def,..."	One word for each part of a value with several numbers: req (must be there), def (can be left out, the engine uses its default) or prev (if left out, the part before is used again).
max	The most parts an intvec or expvec can have.
list	The words allowed for keyword, expkeyword and exp2keyword, separated by commas.
char	Letters allowed before a number: S (the player's own AIR or SND file), F (the common fight effects or sounds) or both (SF).
format="indexed"	The name is followed by a number in brackets: var(5) = 1.
nil="true"	The value can be empty (or -1) to mean "none".
minver	The first engine version that knows the parameter.
deprecated	The version from which the parameter is old.

Kinds of values:

Type	Value
int, float	One number (float also takes whole numbers)
int2 to int4, float2 to float4	2 to 4 numbers separated by commas
ifloat3	A whole number and then three numbers
intvec	1 to max whole numbers
string	Text in double quotes
ustring	Text up to the end of the line, with or without quotes
file	A file name, looked up in the engine's folders (for TrueType fonts also in the system font folder)
keyword	One word from list
keyseq	A command, the keys to press (~D, DF, F, x)
exp	Any expression
expi, expf, expb	An expression that gives a whole number, a decimal number, or 0 or 1
exp2 to exp4, exp2i to exp4i, exp2f to exp4f, exp2b, exp3b	2 to 4 expressions of that kind, separated by commas
expvec	A list of expressions (up to max)
expkeyword	An expression and then a word from list (priority = 4, Hit)
exp2keyword	Two expressions and then a word from list (pos = 0, -10, Head)

Type	Value
hitattr	A hit attribute: state types, then two-letter classes (SCA, NA, SP)
hitflags	Letters from HLAMFD+-
guardflags	Letters from HLAM

In definition files an empty value (intro.storyboard =) is fine for any parameter that isn't required.

Versions

minver and deprecated take WinBeta (the WinMUGEN releases), 1.0 or 1.1, in the order DOS, WinBeta, 1.0, 1.1. (ext is kept for extended engines.) The project's version comes from mugenversion: 1.0 and 1.1 are themselves, and dates like 04,14,2002 mean a WinMUGEN or DOS release, depending on the date. A parameter, block or trigger newer than the project gives a warning. An old one is marked for the deprecation list once the project reaches that version.

Constant lists

<consts name="..."> lists the names that Const() (const), GetHitVar() (GetHitVar), the AssertSpecial flags (AssertSpecial) and StageVar() (StageVar) accept. They are highlighted in the const color.

Triggers

<trigger name="..." [type="..."] [list="..."] [context="..."] [minver="..."] [deprecated="..."]/>

type says how the trigger is written. Without it, the trigger is a plain value (Life, Time).

type	How it's written
exp, exp2, exp3	A function with 1, 2 or 3 expressions: Var(3), Log(2, x), Cond(c, a, b)
keyword	Followed by a part from list: Pos X, Vel Y
valueeq	Compared with = or != to a word from list: StateType = A
opstr	Compared with = or != to a quoted text: Command = "x"
opint	Compared with a whole number. The name can have an ID: ProjHit1000 = 1
opint2	TimeMod = divisor, value
op2int2	AnimElem = n or AnimElem = n, >= t
const	Its argument is checked against the <consts> list of the same name: Const(data.life)
constexp	StageVar(info.name) = "..."
nilexp	The argument is optional: NumHelper or NumHelper(10)
expopt	The argument is optional: IsHelper or IsHelper(10)
hitattr	Followed by a hit attribute: HitDefAttr = SC, NA

A * in a trigger name matches any text (ProjHit* covers ProjHit1000). context says where the trigger can be used. default means character code, the only place M.U.G.E.N has.

Redirections (parent, root, helper(id), target(id), partner, enemy(n), enemynear(n), playerid(id)), operators and the AIR format are built into the code.

completer.xml

```
<completion>
  <model prefix="type=">
    <word value=" ChangeState"/>
  </model>
</completion>
```

When the text you just typed ends with a model's prefix (upper or lower case doesn't matter), the editor offers its words. The word is put in as written, with any space in front or closing bracket. The models that come with the program are type= (controllers), const(, gethitvar(, stagevar(, and flag=, flag2= and flag3= (AssertSpecial flags).

Snippets

One snippet per file, and the file name is the snippet's name.

```
<snippet>
  <field name="sdef" title="StateDef number" type="int"/>
  <field name="stype" title="State type" type="items" value="S,C,A" value_title="Standing,Crouching,Aerial"/>
  <code><![CDATA[[Statedef @sdef]
type = @stype
CURSOR]]></code>
</snippet>
```

Each <field> becomes a box in the insert dialog (int, string, or items with matching value and value_title lists). @name in the code is replaced with what was typed in the field, and the text cursor goes where CURSOR is. Indentation that every line has is removed when the snippet is put in. The snippets that come with the program keep their code at the start of the line inside a CDATA section.

State controller templates

The sctrls/<Name>.sctrl files are plain ASCII with Windows line endings, and are put in as they are. They are made from syntax.xml. After you change a controller, run:

```
python tools/gen_sctrl_templates.py
```

PalFX templates

```
<palfx>
  <add red="150" green="50" blue="-40"/>
  <mul red="256" green="210" blue="120"/>
  <sin red="60" green="30" blue="0" period="8"/>
  <color value="256"/>
  <invert value="false"/>
</palfx>
```

add and sin can be negative, mul uses 256 for 1.0, period is in ticks, color goes from 0 (gray) to 256, and invert is true or false.

Reference pages

The pages are docs/<lang>/<variant>/<kind>.<name>.html, where kind is sctrl or trigger, the name is in lower case, and variant is common, 1.0, 1.1, win (WinMUGEN) or dos. The editor shows the page in the project's version folder if there is one, and the common page if not. Pages are bits of HTML that Qt can show: headings, paragraphs, tables, lists, pre, code, b and i.

Extensions

Things added to the format. All of them are optional:

- raw="true" on a block: the section holds free text and no parameters ([Infobox Text], [Characters], [ExtraStages]).
- The types float3, float4, exp4f and exp3b, named like the others.
- deprecated on triggers (TimeMod).
- The stage background blocks (BGDef, BG>..., BGCtrlDef, BGCtrl>...) and the [Info] and [Files] blocks are in the database. The program has none of them built in.
- A block's own parameter replaces an inherited one with the same name.

Checking your changes

```
python tools/validate_syntax.py
python tools/validate_syntax.py --corpus path/to/chars path/to/stages
```

The first command checks the database and the files made from it (it is also the data_mugen_syntax test). The second one also lists the sections, controllers, parameters and triggers in real files that the database doesn't know. With OFS_CORPUS_KFM and OFS_CORPUS_MUGEN set, ctest -L corpus runs this check on Kung Fu Man and on the data, stages and font folders of a M.U.G.E.N install, and fails if anything is unknown.

To add a controller: add its State>Name block to the state file type and a matching State>Name:state.Name block to command, add its name to the type= model in completer.xml, write docs/en-us/common/sctrl.<name>.html, make the templates again and run the checker.